哈尔滨工业大学（深圳）

实验报告

开课学期： 2026 年春季
课程名称： 网络安全攻防竞赛实践
实验名称： Reverse 逆向工程实验
实验时间： 2026.4.24
学生班级： 电气工程及其自动化 1 班
学生学号： 2024312506
学生姓名： 余泓彬

1 题目 1：例题-算法

1.1 题目分析

本题为逆向工程入门例题，核心考察 PE 文件分析、静态反汇编、异或算法逆向的基础流程。程序会对用户输入的字符串逐字节与常量 1 进行异或运算，再与硬编码的密文进行比对，完全匹配则输出 flag。解题关键在于利用异或运算的可逆性，通过密文还原明文 flag。

1.2 解题步骤

1. 使用 Detect It Easy (DiE) 工具分析程序：确认文件为 64 位 Windows 平台 PE 可执行文件，无壳，可直接进行静态分析。
2. 使用 IDA Pro 64 位加载程序，进入汇编视图查看主函数控制流。
3. 按 F5 查看伪代码，发现核心校验逻辑：程序循环读取用户输入，逐字节与 1 异或后，与字符串常量进行比较。
4. 双击字符串常量，定位到数据段中的密文。
5. 根据异或运算的可逆性，将密文字符串的每个字节与 1 异或，即可还原出明文 flag。
6. 编写 Python 脚本完成解密，得到最终 flag。

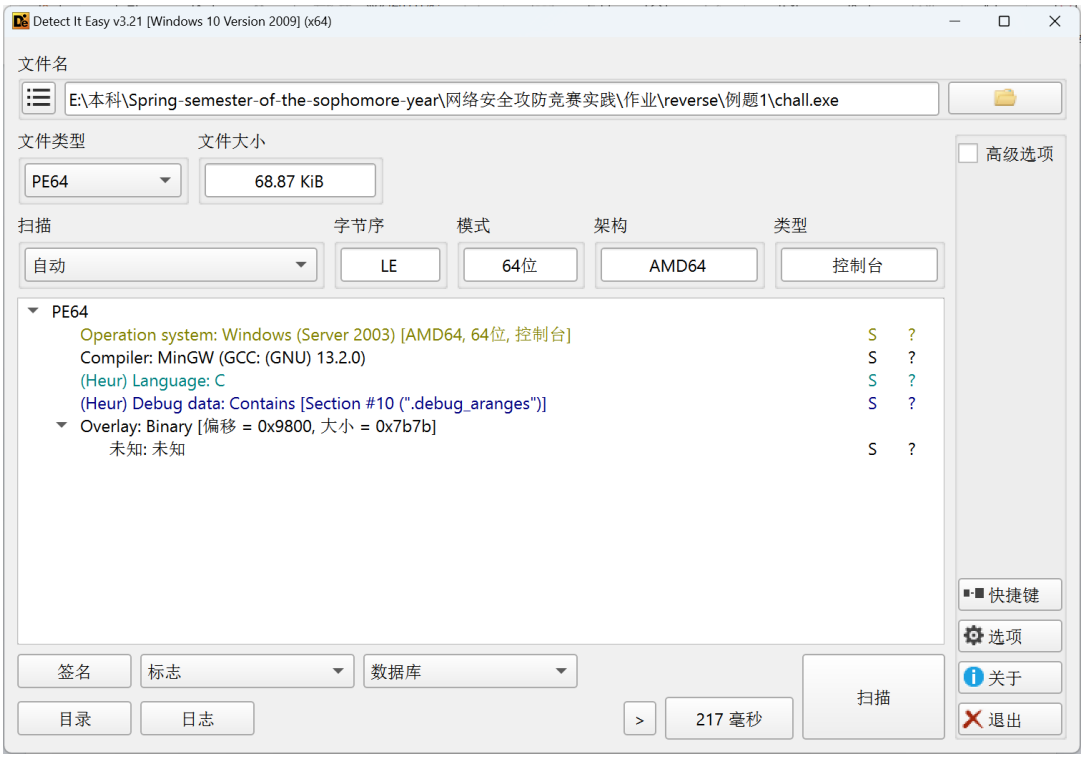


图 1: 使用 Detect It Easy 确认程序为 64 位无壳 PE 文件

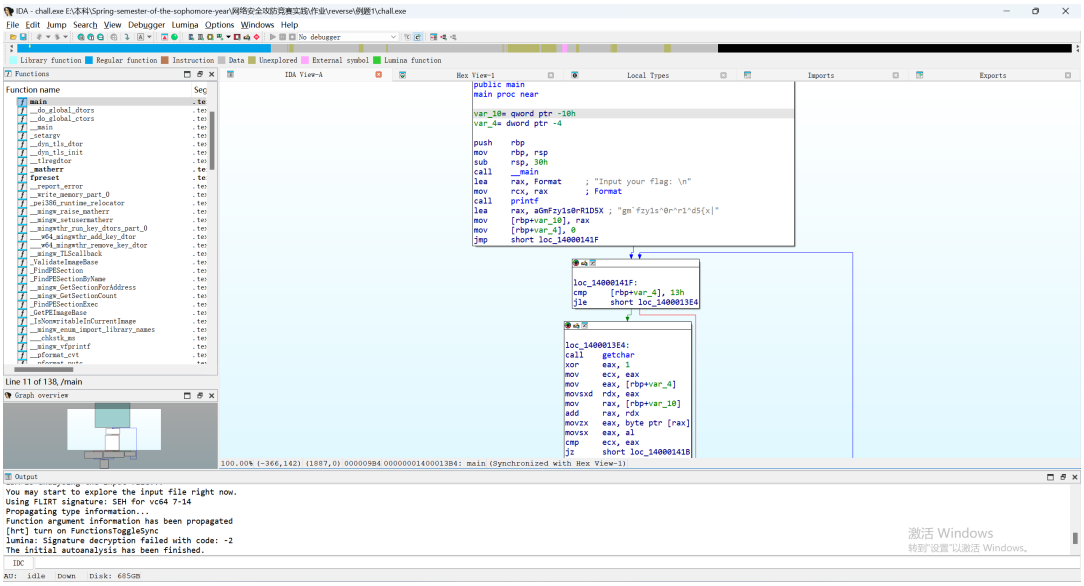


图 2: IDA Pro 中查看主函数汇编代码

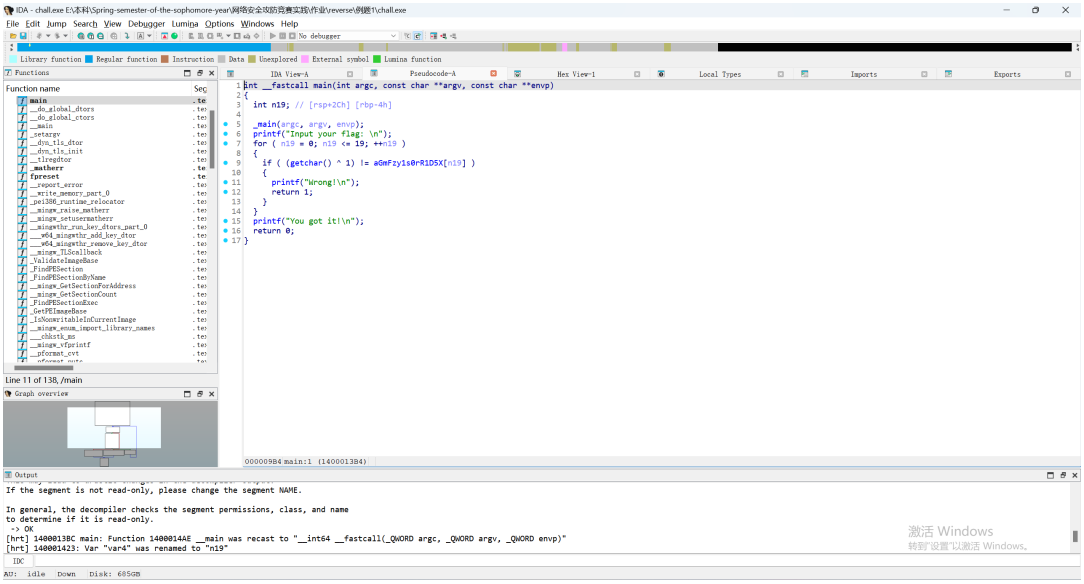


图 3: IDA 中查看主函数伪代码，发现异或校验逻辑

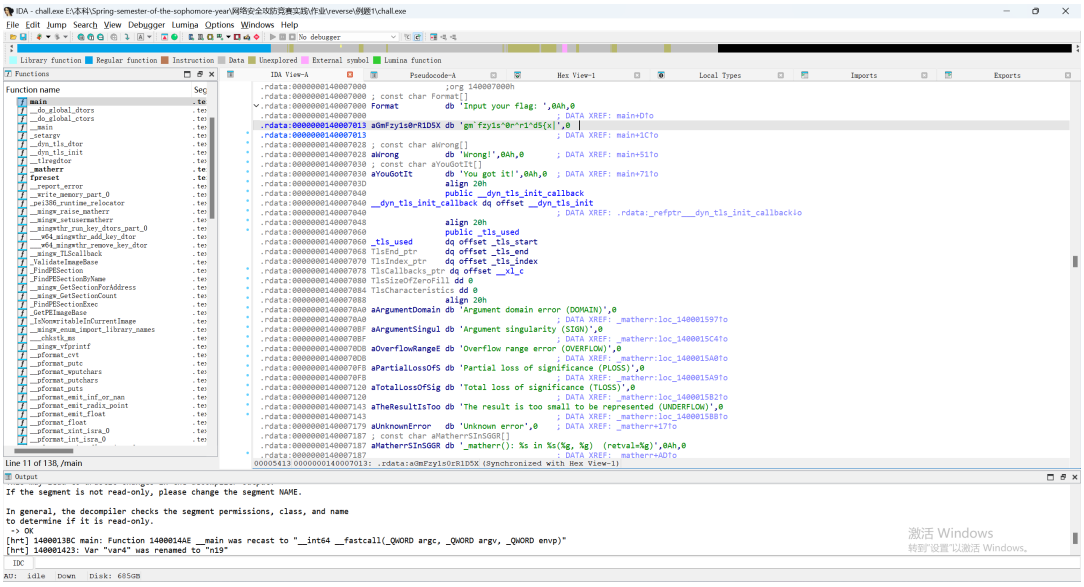
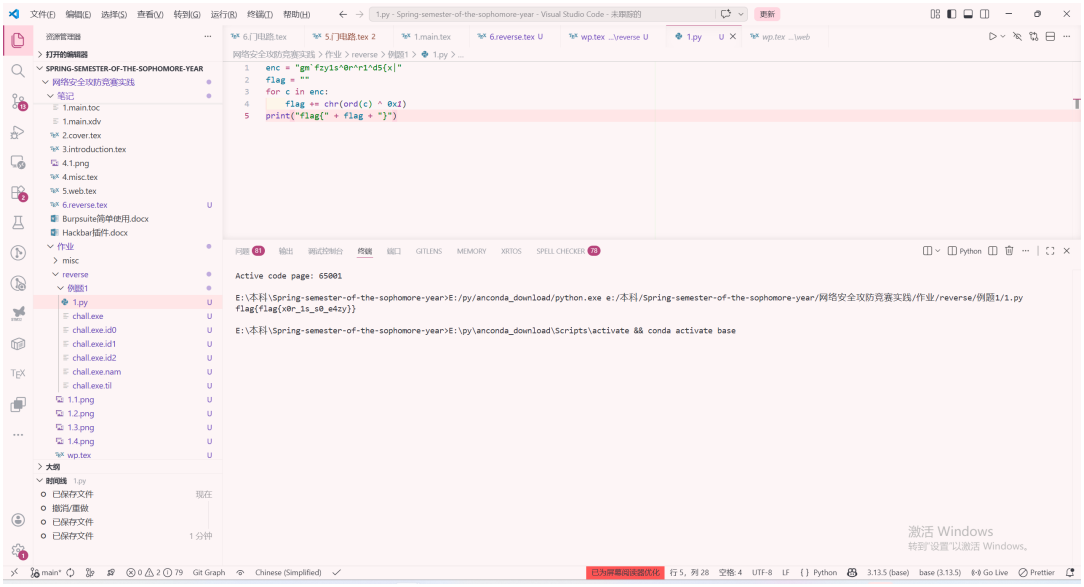


图 4: IDA 数据段中查看密文字符串常量

1.2.1 解题脚本与运行结果

```
enc = "gm`fzy1s^0r^r1^d5{x|"
flag = ""
for c in enc:
    flag += chr(ord(c) ^ 0x1)
print("flag{" + flag + "}")
```



1.3 最终 Flag

$$\text{flag}\{\text{fl0r1s_e4zy_is_x0r}\}$$

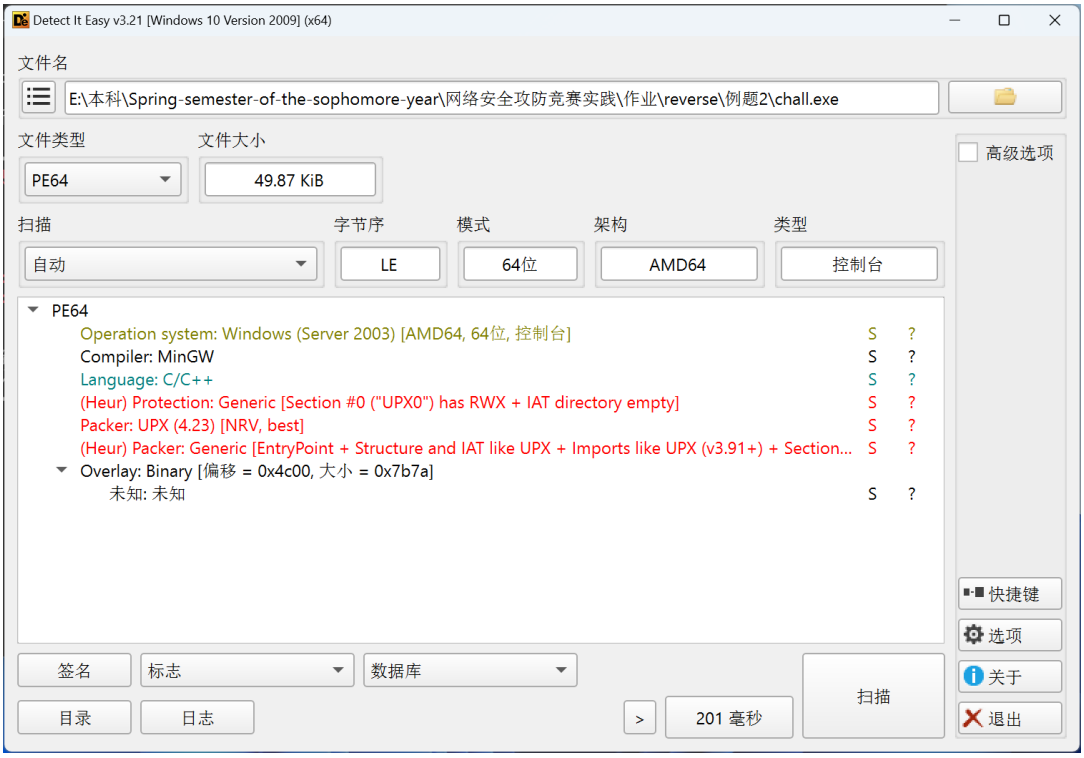
2 题目 2：例题-UPX 脱壳

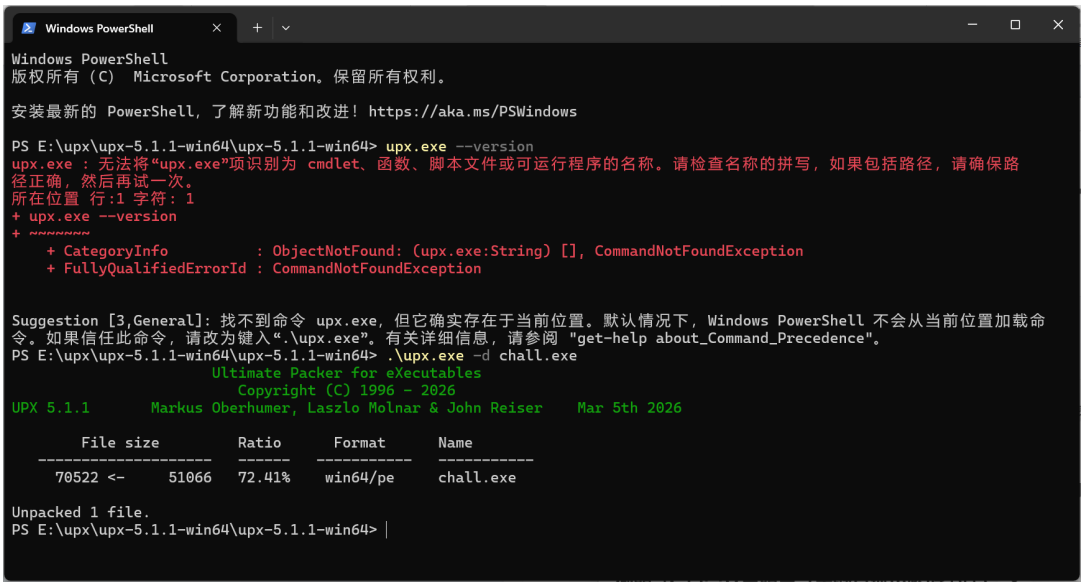
2.1 题目分析

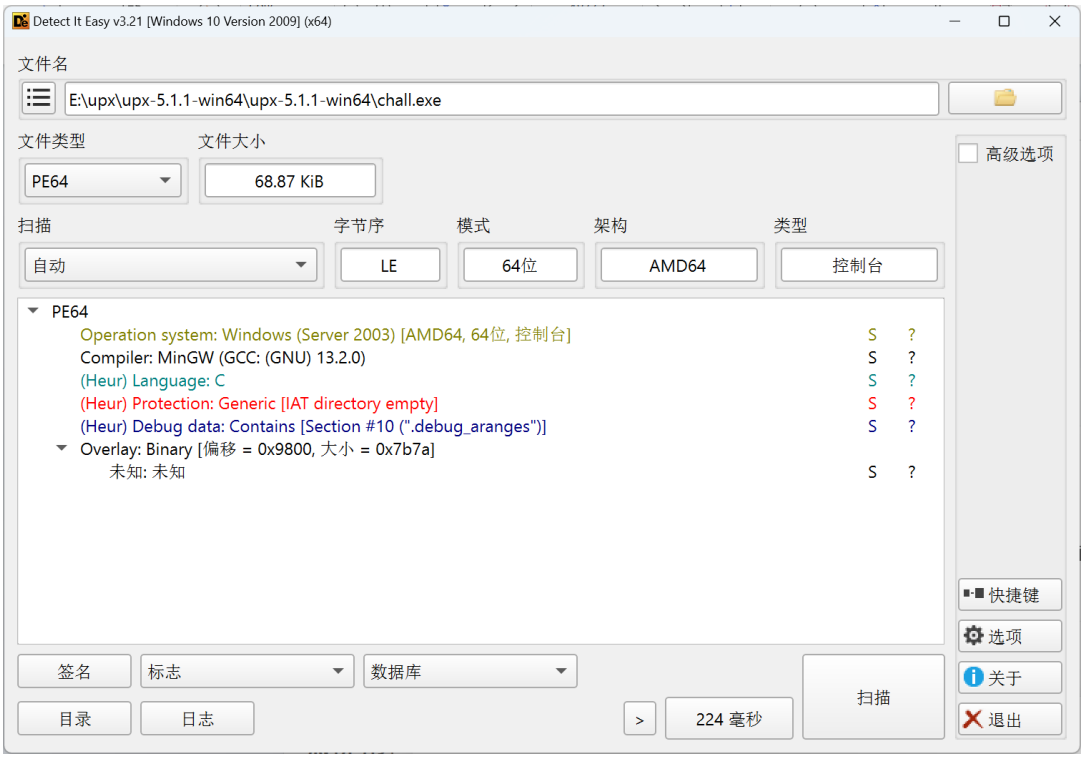
本题为逆向工程中典型的 UPX 壳脱壳例题，核心考察 PE 文件壳识别、UPX 工具手动脱壳、脱壳后程序静态分析的完整流程。程序被标准 UPX 壳压缩，直接反汇编无法查看有效代码，需先脱壳再分析，脱壳后可直接从主函数中找到明文 flag。

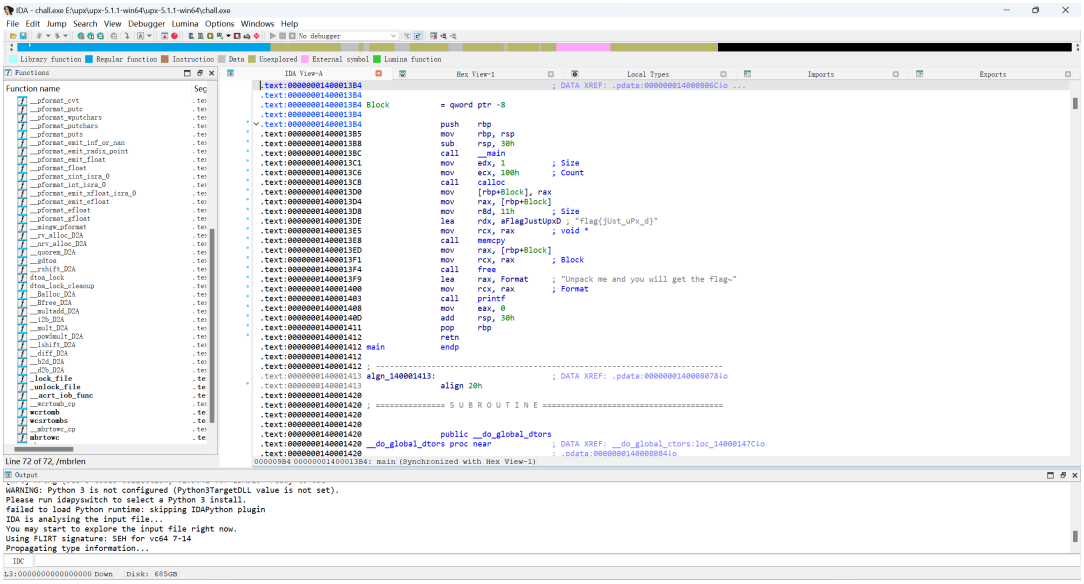
2.2 解题步骤

1. 使用 Detect It Easy (DiE) 工具分析 `chall.exe`，确认文件为 64 位 PE 可执行文件，被 UPX(4.23)[NRV.best] 壳压缩。
2. 使用 UPX 工具执行脱壳命令：`upx.exe -d chall.exe`，完成自动脱壳。
3. 脱壳完成后，再次使用 DiE 扫描文件，确认 UPX 壳已被移除，程序恢复为无壳状态。
4. 使用 IDA Pro 64 位加载脱壳后的程序，直接查看主函数汇编代码，即可找到明文 flag 字符串。




Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。
安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows
PS E:\upx\upx-5.1.1-win64\upx-5.1.1-win64> upx.exe --version
upx.exe : 无法将“upx.exe”项识别为 cmdlet、函数、脚本文件或可运行程序的名称。请检查名称的拼写，如果包括路径，请确保路径正确，然后再试一次。
所在位置 行:1 字符: 1
+ upx.exe --version
+ ~~~~~~~
+ CategoryInfo : ObjectNotFound: (upx.exe:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
Suggestion [3,General]: 找不到命令 upx.exe，但它确实存在于当前位置。默认情况下，Windows PowerShell 不会从当前位置加载命令。如果信任此命令，请改为键入“.\upx.exe”。有关详细信息，请参阅 "get-help about_Command_Precedence"。
PS E:\upx\upx-5.1.1-win64\upx-5.1.1-win64> .\upx.exe -d chall.exe
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2026
UPX 5.1.1 Markus Oberhumer, Laszlo Molnar & John Reiser Mar 5th 2026
File size Ratio Format Name
70522 <- 51066 72.41% win64/pe chall.exe
Unpacked 1 file.
PS E:\upx\upx-5.1.1-win64\upx-5.1.1-win64>


Detect It Easy v3.21 [Windows 10 Version 2009] (x64)
文件名
E:\upx\upx-5.1.1-win64\upx-5.1.1-win64\chall.exe
文件类型
文件大小
PE64
68.87 KiB
高级选项
扫描
字节序
模式
架构
类型
自动
LE
64位
AMD64
控制台
PE64
Operation system: Windows (Server 2003) [AMD64, 64位, 控制台]
Compiler: MinGW (GCC: (GNU) 13.2.0)
(Heur) Language: C
(Heur) Protection: Generic [IAT directory empty]
(Heur) Debug data: Contains [Section #10 (".debug_aranges")]
Overlay: Binary [偏移 = 0x9800, 大小 = 0x7b7a]
未知: 未知
快捷键
选项
关于
退出
签名
标志
数据库
扫描
目录
日志
224 毫秒



2.3 最终 Flag

$$\text{flag\{jUst_uPx_d\}}$$

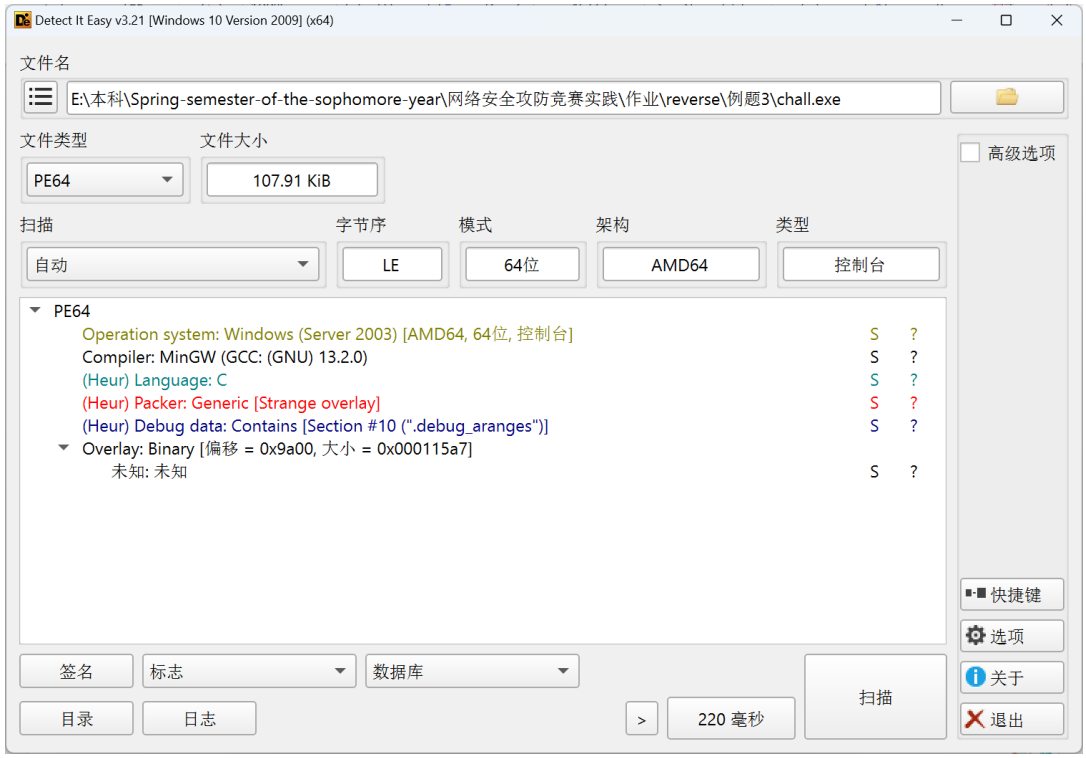
3 题目 3：例题-自定义壳脱壳

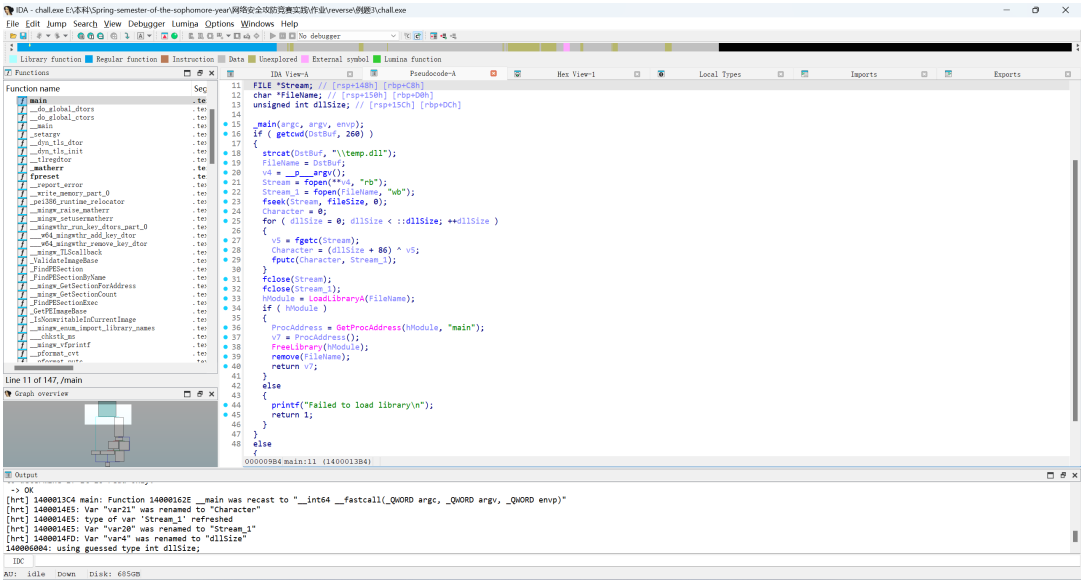
3.1 题目分析

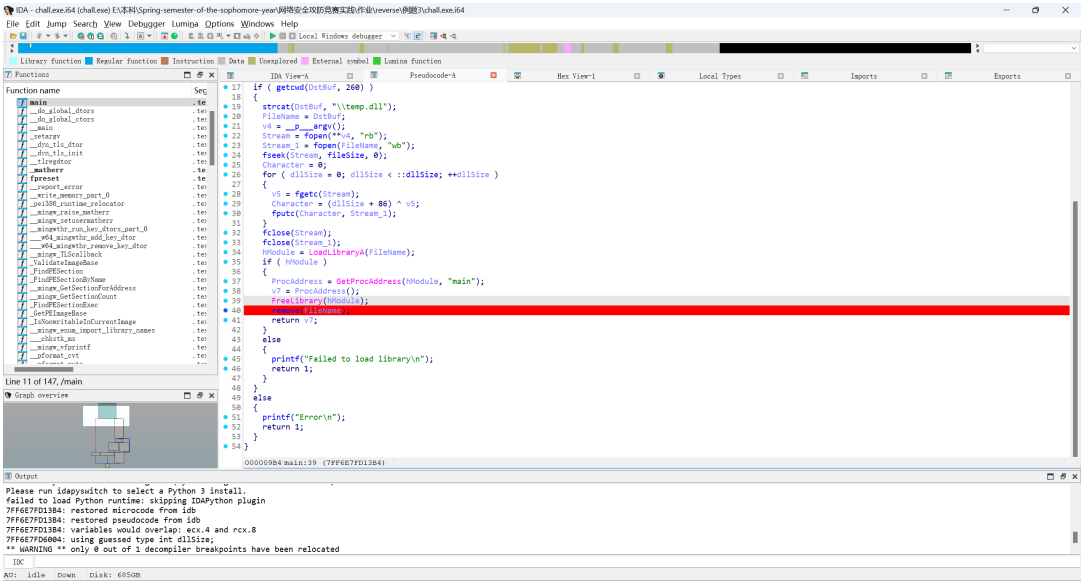
本题属于自定义壳动态加载逆向，无常规 PE 壳特征，静态分析无法得到有效逻辑。程序运行时会解密释放临时 DLL 文件，执行完核心逻辑后调用删除接口清理临时文件，需要借助 IDA 动态调试断点拦截，在文件删除前抓取 DLL 再静态分析获取 flag。

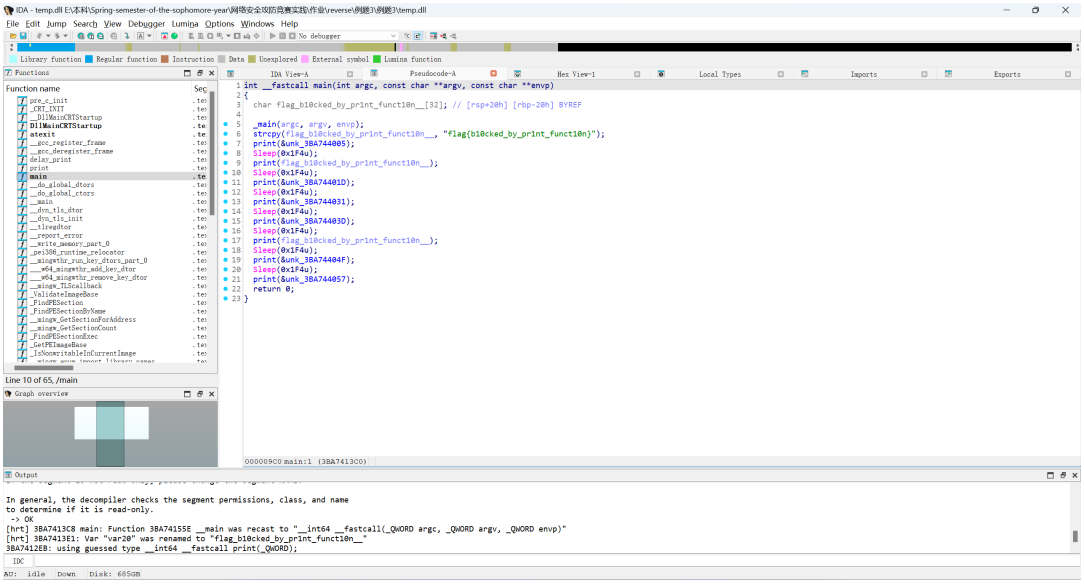
3.2 解题步骤

1. 使用 Detect It Easy 检测程序，无标准 UPX 壳及常见壳特征，判定为自定义壳保护。
2. IDA 载入程序，F5 反编译伪代码，观察到程序生成并加载 `.temp.dll`，最后调用 `remove` 删除临时文件。
3. 在 `remove(FileName)` 语句行设置断点，选择本地 Windows 调试器启动调试。
4. 程序断下时，临时 DLL 已生成尚未删除，立刻复制备份该 DLL 文件。
5. 使用 IDA 载入备份的 `.temp.dll`，静态分析即可得到明文 flag。




IDA - chall.exe E:\本科\Spring-semester-of-the-sophomore-year\网络安全攻防竞赛实践\作业\reverse\例题3\chall.exe
File Edit Jump Search View Debugger Lumina Options Windows Help
Functions
Function name
IDA View-A
Pseudocode-A
Hex View-1
Local Types
Imports
Exports
Line 11 of 147, /main
Graph overview
Output
-> OK
[hrt] 1400013C4 main: Function 14000162E __main was recast to "__int64 __fastcall(_QWORD argc, _QWORD argv, _QWORD envp)"
[hrt] 1400014E5: Var "var21" was renamed to "Character"
[hrt] 1400014E5: type of var 'Stream_1' refreshed
[hrt] 1400014E5: Var "var20" was renamed to "Stream_1"
[hrt] 1400014FD: Var "var4" was renamed to "dllSize"
140006004: using guessed type int dllSize;
IDC
AU: idle Down Disk: 685GB


IDA - chall.exe64 (chall.exe) E:\本科\Spring-semester-of-the-sophomore-year\网络安全攻防竞赛实践\作业\reverse\例题3\chall.exe64
File Edit Jump Search View Debugger Lumina Options Windows Help
Functions
Function name
IDA View-A
Pseudocode-A
Hex View-1
Local Types
Imports
Exports
Line 11 of 147, /main
Graph overview
Output
Please run idapyswitch to select a Python 3 install.
failed to load Python runtime: skipping IDAPython plugin
7FF6E7FD13B4: restored microcode from idb
7FF6E7FD13B4: restored pseudocode from idb
7FF6E7FD13B4: variables would overlap: ecx.4 and rcx.8
7FF6E7FD6004: using guessed type int dllSize;
** WARNING ** only 0 out of 1 decompiler breakpoints have been relocated
IDC
AU: idle Down Disk: 685GB


IDA - temp.dll E:\本科\Spring-semester-of-the-sophomore-year\网络安全攻防竞赛实践\作业\reverse\例题3\temp.dll
File Edit Jump Search View Debugger Lumina Options Windows Help
Functions
Function name
IDA View-A
Pseudocode-A
Hex View-1
Local Types
Imports
Exports
Line 10 of 65, /main
Graph overview
Output
In general, the decompiler checks the segment permissions, class, and name
to determine if it is read-only.
-> OK
[hrt] 3BA7413C8 main: Function 3BA74155E __main was recast to "__int64 __fastcall(_QWORD argc, _QWORD argv, _QWORD envp)"
[hrt] 3BA7413E1: Var "var20" was renamed to "flag_b10cked_by_print_funct10n__"
3BA7412EB: using guessed type __int64 __fastcall print(_QWORD);
IDC
AU: idle Down Disk: 685GB

3.3 最终 Flag

flag{b10cked_by_pr1nt_funct10n}